



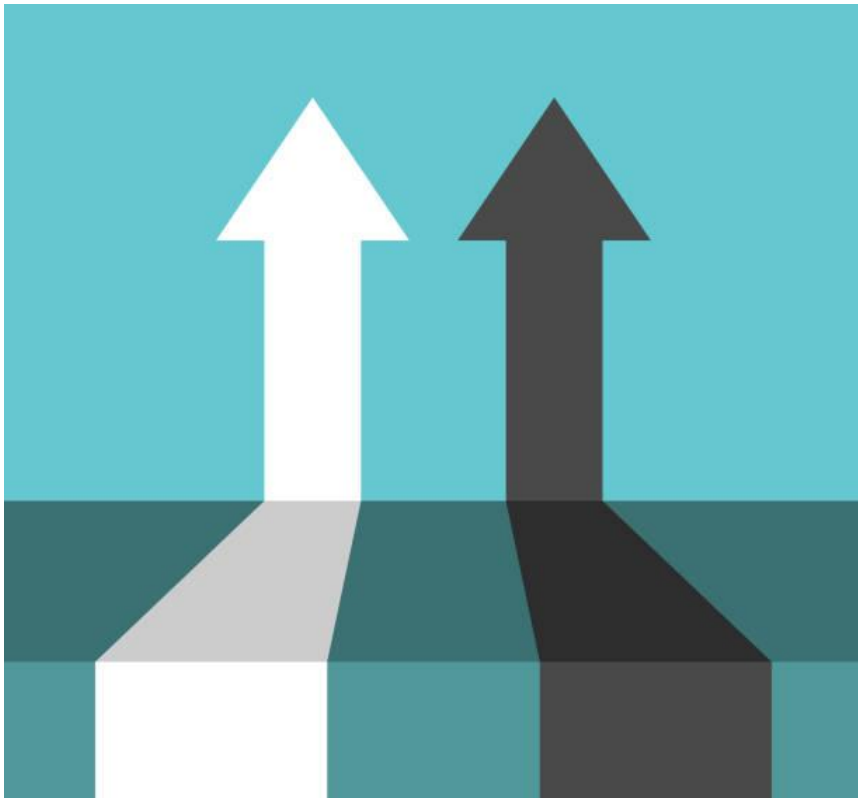
Cómputo Paralelo

y en la Nube

Profesor:

Dr. J. Octavio Gutiérrez García

octavio.gutierrez@itam.mx





OpenMP

Open specifications for Multi-Processing



C++

Conceptos ultra básicos

Instalar C/C++ y Configurar VSCode

<https://code.visualstudio.com/docs/cpp/config-mingw>

Agrega al Path la ruta de los compiladores:.

Por ejemplo: C:\msys64\ucrt64\bin

Terminal > Configure Default Build Task.

De las opciones elige **C/C++: g++ build active file.**

En los argumentos de task.json agrega "-fopenmp"

```
g++ holamundo.cpp -o holamundo -fopenmp
```

En caso de tener problemas con su instalación en Windows 11:

How to Install C and C++ compiler (gcc g++ gdb) on Windows 11 using mingw and msys2

<https://youtu.be/-gxwT-eAfvU?si=czGzB4YAec8F0otG>



Hello World C

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello, World!");  
    return 0;  
}
```



Hello World C++

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello";  
    return 0;  
}
```



Hello World C++

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    cout << "Hello";  
    return 0;  
}
```



**Preparándose para
paralelizar operaciones:**

Arreglos dinámicos con C++

C++: conceptos no tan básicos

- Referencias
- Apuntadores
- **Reservar** memoria para arreglos de 1 y 2 dims
 - **Liberar memoria** de arreglos
- Pasar argumentos en C++



C++: Apuntadores

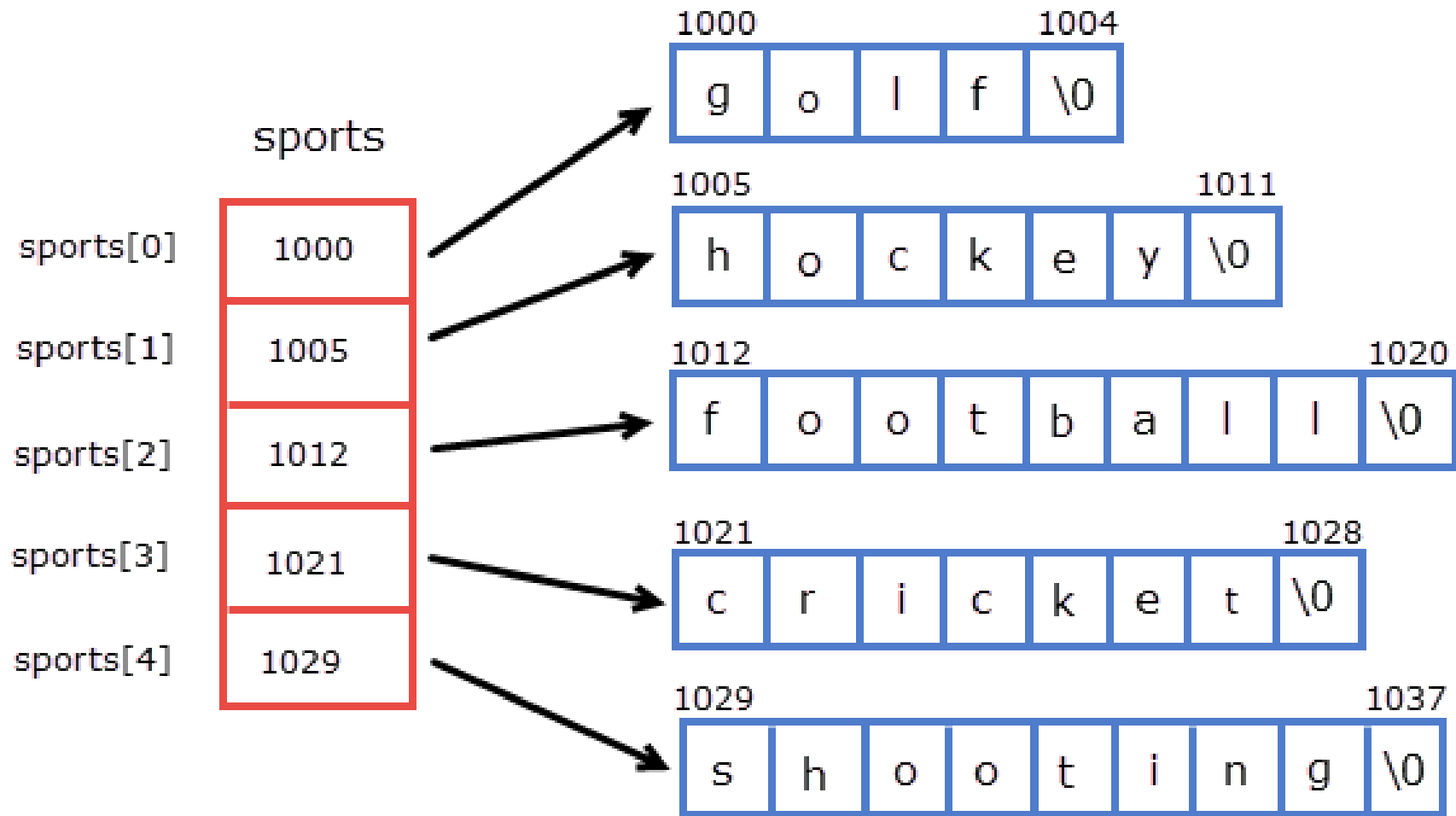
**Creación
Dinámica de
Arreglo**

```
// Reservar memoria para el arreglo
int* arreglo {new int[5]{1, 2, 3, 4, 5}};

/* Tu Código - Tu Código - Tu Código */

// Liberar memoria previamente asignada
delete[] arreglo;
```

C++: Apuntadores



C++: Apuntadores

```
int col = 4;  
int ren = 2;
```

```
// Reservar memoria para la matriz  
int** matriz {new int*[ren]};  
for (int i = 0; i < ren; i++){  
    matriz[i] = new int[col];  
}
```

```
/* Tu Código - Tu Código - Tu Código - Tu Código - Tu Código - */
```

```
// Liberar la memoria previamente asignada  
for (int i = 0; i < ren; i++){  
    delete[] matriz[i];  
}  
delete[] matriz;
```

**Creación
Dinámica de
Matriz**

C++: Apuntes

Matriz

1	2	3	4
5	6	7	8

*matriz -> 0x219f2584**e50**

matriz[0] -> 0x219f2584**e50**

0x219f2584**e50** 0x219f2584**e54** 0x219f2584**e58** 0x219f2584**e5c**

matriz[1] -> 0x219f2584**f00**

0x219f2584**f00** 0x219f2584**f04** 0x219f2584**f08** 0x219f2584**f0c**

```
cout << *matriz << "\n";
for (int i = 0; i < ren; i++) {
    cout << matriz[i] << "\n";
    for (int j = 0; j < col; j++){
        cout << &matriz[i][j] << " ";
    }
    cout << "\n";
}
```

Pasando argumentos en C++

```
#include <iostream>
using namespace std;
```

```
int main(int argc, char** argv) {
    if (argc == 1) {
        cout << "No se pasaron argumentos " << argv[0];
        return 0;
    } else {
        cout << " Ingresaste " << argc - 1 << " argumentos" << "\n";
        for (int i = 1; i < argc; i++) {
            cout << argv[i] << "\n";
        }
    }
    return 0;
}
```

atoi()
atol()
atof()

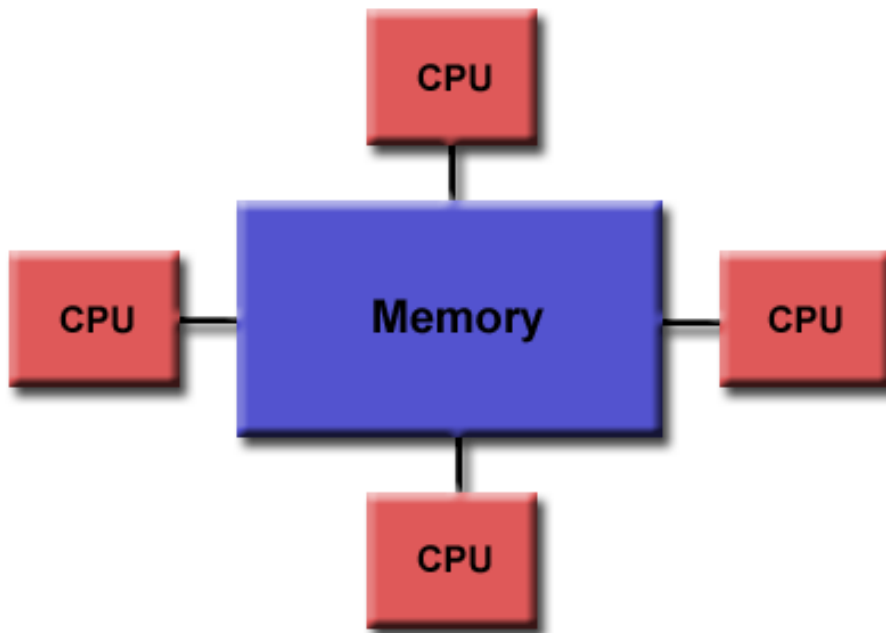


OpenMP

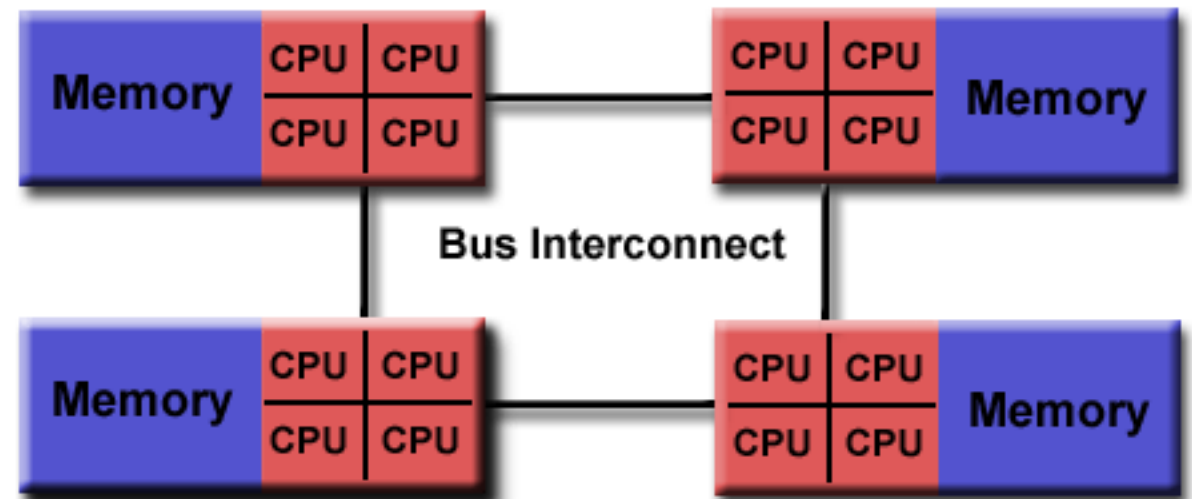
Open specifications for Multi-Processing

OpenMP diseñado para Memoria Compartida

UMA



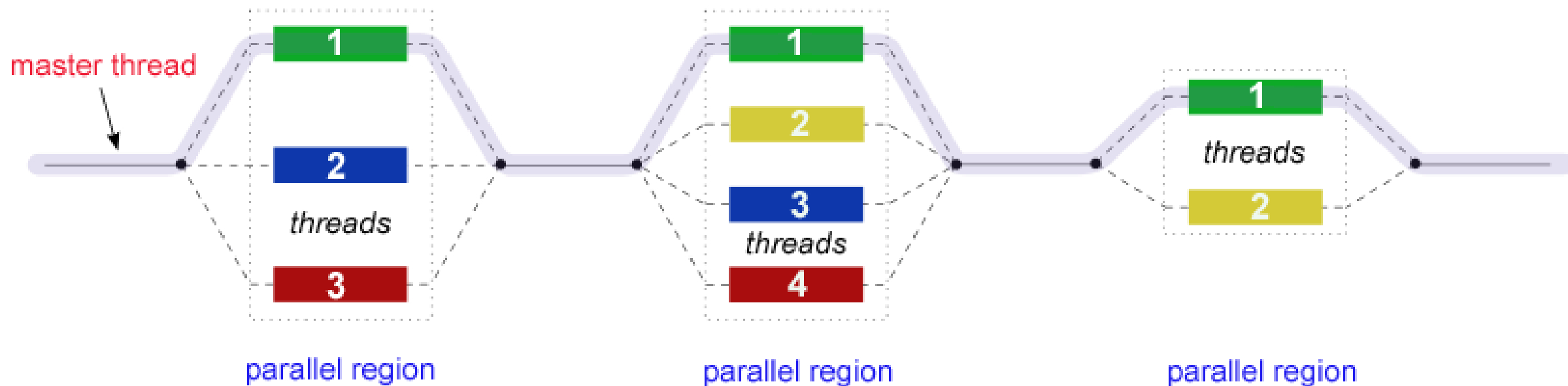
NUMA



OpenMP

Enfoque Multi-hilo

Modelo Fork-Join de Ejecución Paralela



OpenMP

- *Single Program, Multiple Data* (subcategoría de MIMD)
- Aplicaciones paralelas con *memoria compartida*
- *Control total* - paralelismo explícito (no automático)
- Mecanismos de *sincronización*
- Número *dinámico* de hilos
- Cada hilo puede tener variables *compartidas y locales*

The word "details" is written in a playful, hand-drawn style using various colors (blue, red, green, yellow, pink) and thick, textured strokes. The letters are slightly tilted and have a 3D effect with shadows.

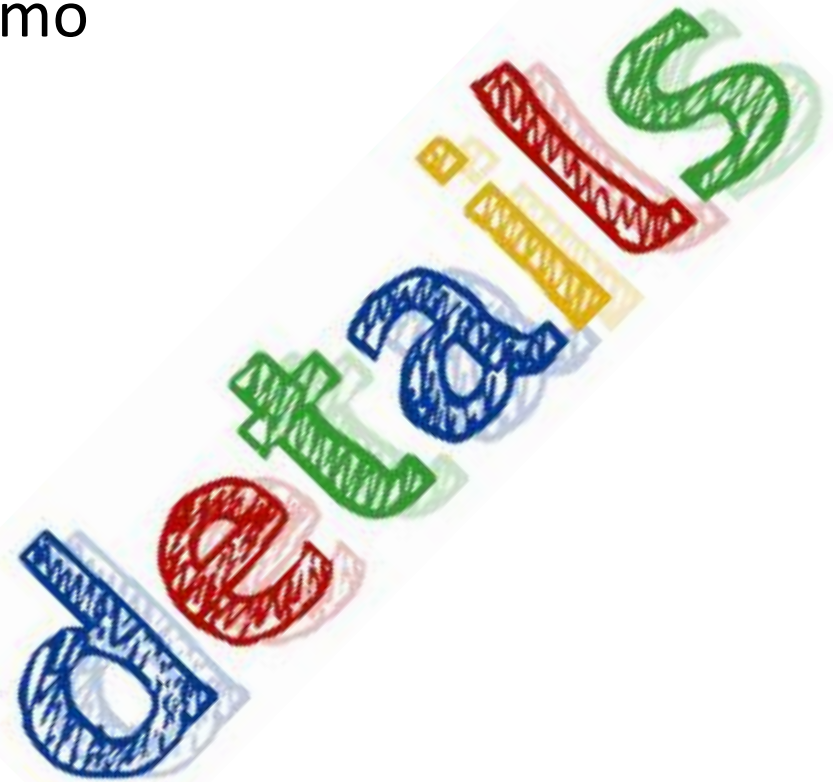
OpenMP

- Hilos tienen potencial de ejecutar el mismo código, sin embargo, cada hilo puede acceder diferentes datos y atravesar *diferentes rutas de ejecución*.

- *Metas*

- Estandarización
- Programación paralela simple
- “Portable”

- Última versión - Noviembre 2024



OpenMP (Open specifications for Multi-Processing)

- *Ofrece*

- Métodos para iniciar hilos paralelos/concurrentes
- Descubrir cuántos hilos están corriendo
- Métodos para unir hilos cuando se requiere ejecución serial
- Métodos para sincronizar hilos

- *No ofrece*

- Métodos para checar dependencias de datos
- Conflictos con datos
- Condiciones de Carrera
- Deadlocks



OpenMP – Directivas de Compilador



- Generación de una región paralela
- División de código/instrucciones entre hilos
- Distribuir iteraciones entre hilos
- Serializar secciones de código
- Sincronización de trabajo entre hilos

#*pragma omp* construct *private*(var1, var2) *shared*(var3)



Formato de las directivas

#pragma omp constructo [cláusula, ...]

Sensibles al uso de mayúsculas y minúsculas

OpenMP - Librerías

```
#include <omp.h>
```

```
int omp_get_num_threads(void)
```

- Ajustar y consultar el número de hilos
- Consultar el id único de un hilo
- Verificar si se encuentra en una región paralela
- Manejo de candados
- Consultar tiempo de reloj de pared

OpenMP - Variables de ambiente

- Ajustar el número de hilos
- Especificar cómo se dividen las iteraciones entre los hilos
- Vincular hilos a procesadores
- Controlar niveles de anidamiento de paralelismo
- (Des)Habilitar hilos dinámicos
- Ajustar el tamaño del stack

export OMP_NUM_THREADS=8



AMBIENT

Tamaño de la memoria Stack

Compiler	Approx. Stack Limit	Approx. Array Size (doubles)
Linux icc, ifort	4 MB	700 x 700
Linux pgcc, pgf90	8 MB	1000 x 1000
Linux gcc, gfortran	2 MB	500 x 500

```
setenv OMP_STACKSIZE 2000500B
setenv OMP_STACKSIZE "3000 k "
setenv OMP_STACKSIZE 10M
setenv OMP_STACKSIZE " 10 M "
setenv OMP_STACKSIZE "20 m "
setenv OMP_STACKSIZE " 1G"
setenv OMP_STACKSIZE 20000
```

Open MP - Estructura general de un programa

```
#include <omp.h>
```

```
int main () {  
    int var1, var2, var3;  
    // Código serial  
    #pragma omp parallel private(var1, var2) shared(var3)  
    {  
        // Ejecución de código por los hilos  
        // Todos los hilos se unen al maestro y se disuelven  
    }  
    // Código serial  
}
```

Región paralela (ejecutada por múltiples hilos) - Directiva

```
#pragma omp parallel [clause ...] newline  
    if (scalar_expression)  
    private (list)  
    shared (list)  
    default (shared | none)  
    firstprivate (list)  
    num_threads (integer-expression)
```

structured_block

Cómo definir el número de hilos

Uso de la función `omp_set_num_threads()`

Ajustando la variable de ambiente `OMP_NUM_THREADS`

Default: usualmente el número de cores virtuales

Los hilos están numerados de 0 (hilo maestro) a N-1

Algunas funciones omp

int	omp_get_thread_num()
int	omp_get_num_threads()
void	omp_set_num_threads(number_threads)
double	omp_get_wtime()



Mi Primer Programa Paralelo con OpenMP

Mi Primer Programa Paralelo con OpenMP

```
#include <omp.h>
#include <iostream>

int main() {
    int nthreads, thread_id;
    /* Crea un equipo de hilos con una variable privada thread_id*/
    #pragma omp parallel private(thread_id)
    {
        /* Obtiene el identificador y lo imprime */
        thread_id = omp_get_thread_num();
        std::cout << "Hola desde el hilo " << thread_id << "\n";

        /* Solo el maestro ejecuta esta instrucción */
        if (thread_id == 0) {
            nthreads = omp_get_num_threads();
            std::cout << "Número de hilos: " << nthreads << "\n";
        }
    } /* Todos los hilos se unen al maestro y terminan */
}
```



Fors Paralelos

Constructos para compartir trabajo

#pragma omp **parallel for**

#pragma omp **for schedule**(*dynamic*, chunk_size)

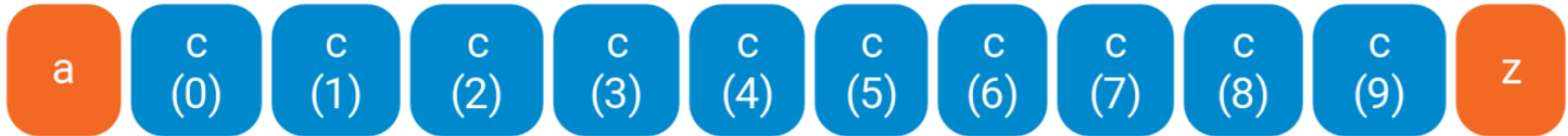
#pragma omp **for schedule**(*static*, chunk_size)

#pragma omp **parallel for** collapse(num_fors)

For simple

```
a();  
for (int i = 0; i < 10; ++i) {  
    c(i);  
}  
z();
```

thread 0:



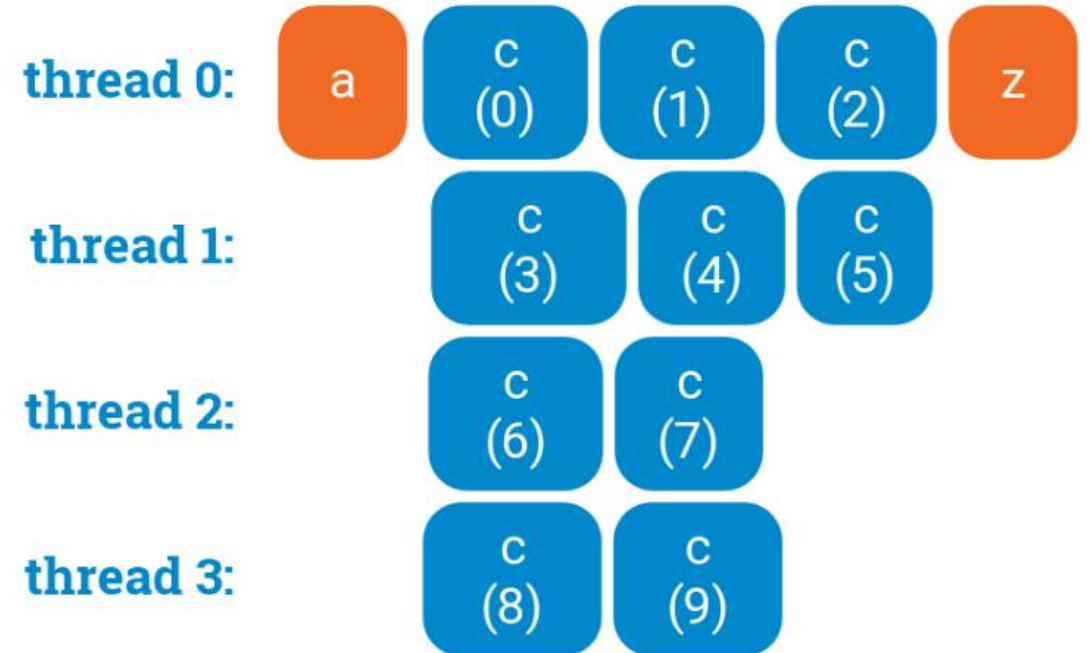
thread 1:

thread 2:

thread 3:

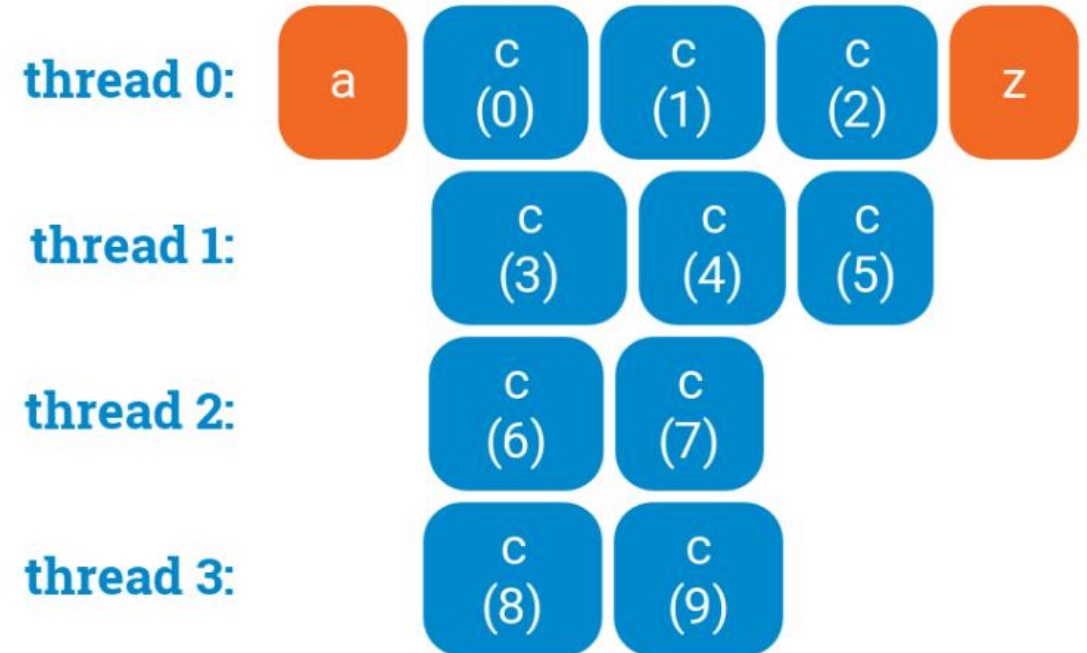
#pragma omp parallel for

```
a();  
#pragma omp parallel for  
for (int i = 0; i < 10; ++i) {  
    c(i);  
}  
z();
```



#pragma omp for

```
a();  
#pragma omp parallel  
{  
    #pragma omp for  
    for (int i = 0; i < 10; ++i) {  
        c(i);  
    }  
}  
z();
```



Errores comunes

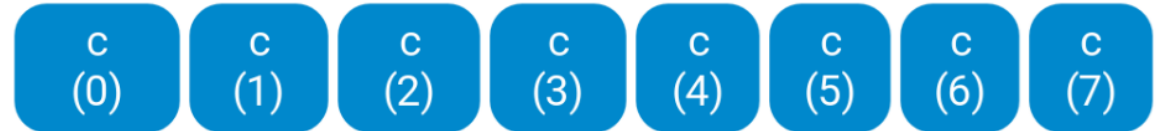


```
a();  
#pragma omp parallel  
for (int i = 0; i < 8; ++i) {  
    c(i);  
}  
z();
```

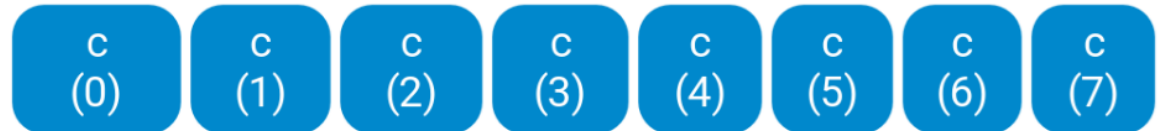
thread 0:



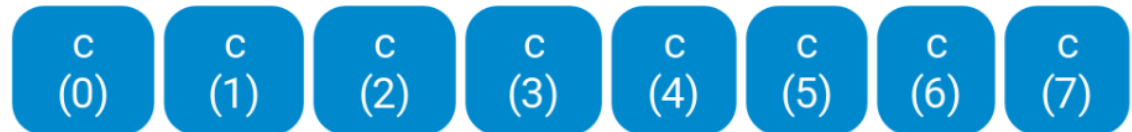
thread 1:



thread 2:



thread 3:



Errores comunes



```
a();
```

```
#pragma omp for
```

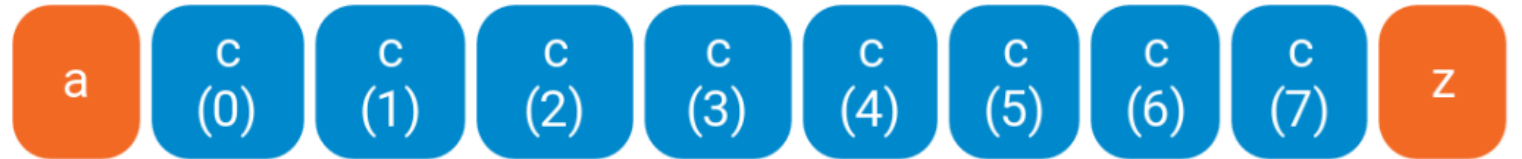
```
for (int i = 0; i < 8; ++i) {
```

```
    c(i);
```

```
}
```

```
z();
```

thread 0:



thread 1:

thread 2:

thread 3:

Ejercicio

float a[N], b[N], c[N]

Dos arreglos float ***a*** y ***b*** con la siguiente inicialización

```
for (i = 0; i < vector_size; i++)  
    a[i] = b[i] = i * 1.0;
```

Paralelizar la suma de los arreglos ***a*** y ***b*** en un tercer arreglo ***c***

```
for (i = 0; i < vector_size; i++)  
    c[i] = a[i] + b[i];
```

for schedule *static*

```
a();
```

```
#pragma omp parallel for
```

```
for (int i = 0; i < 16; ++i) {
```

```
    w(i);
```

```
}
```

```
z();
```



thread 0:



thread 1:



thread 2:

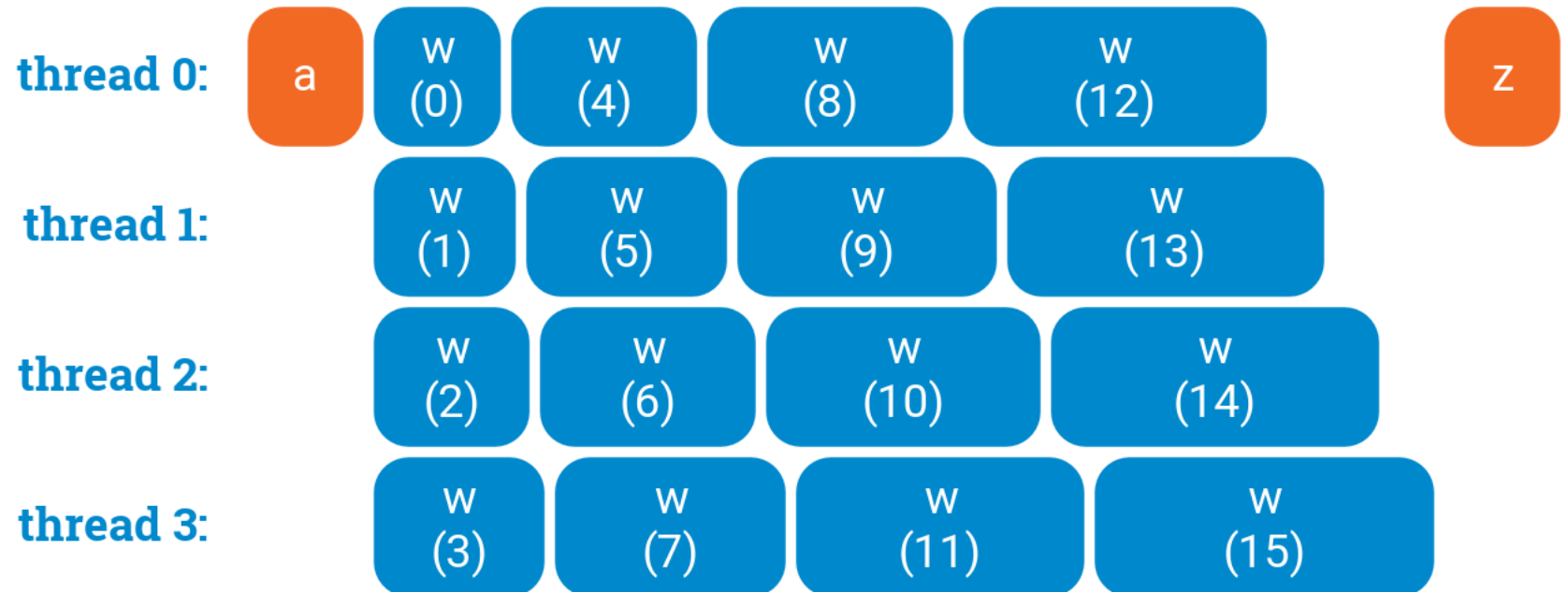


thread 3:



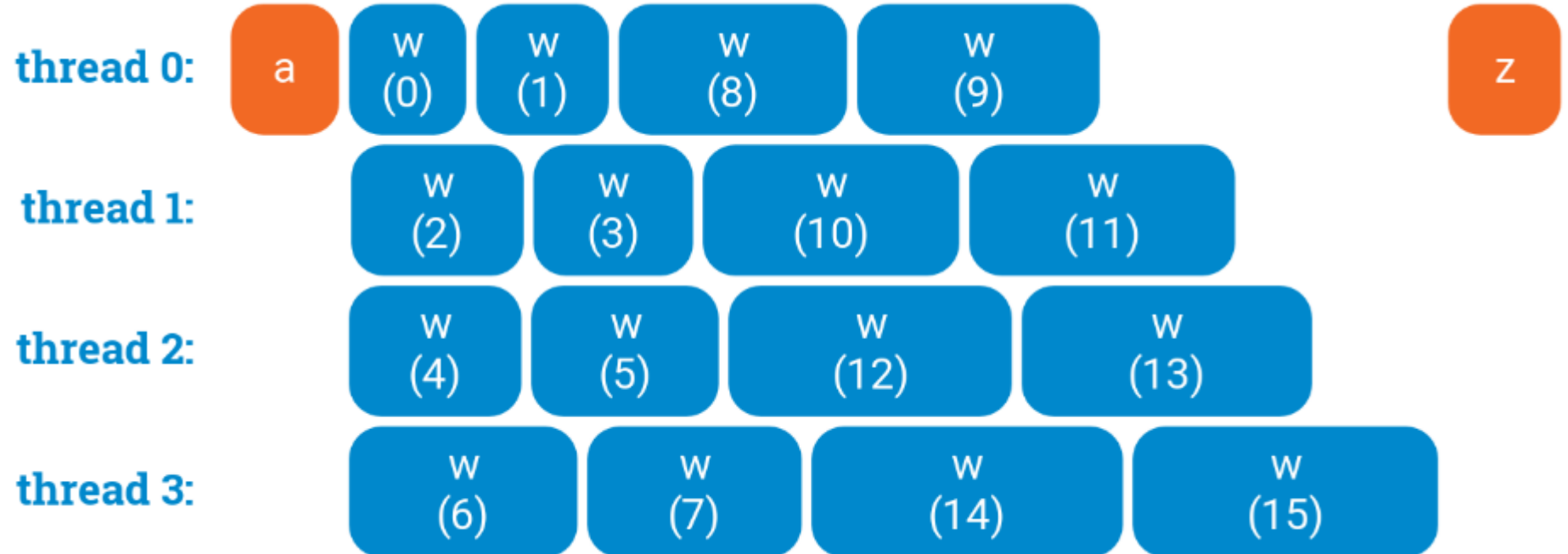
for schedule *static*

```
a();  
#pragma omp parallel for schedule(static,1)  
for (int i = 0; i < 16; ++i) {  
    w(i);  
}  
z();
```



for schedule *static*

```
a();  
#pragma omp parallel for schedule(static,2)  
for (int i = 0; i < 16; ++i) {  
    w(i);  
}  
z();
```



for schedule *dynamic*

```
a();
```

```
#pragma omp parallel for
```

```
for (int i = 0; i < 16; ++i) {
```

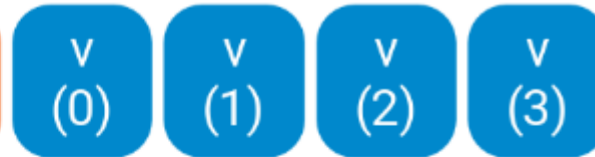
```
    v(i);
```

```
}
```

```
z();
```



thread 0:



thread 1:



thread 2:

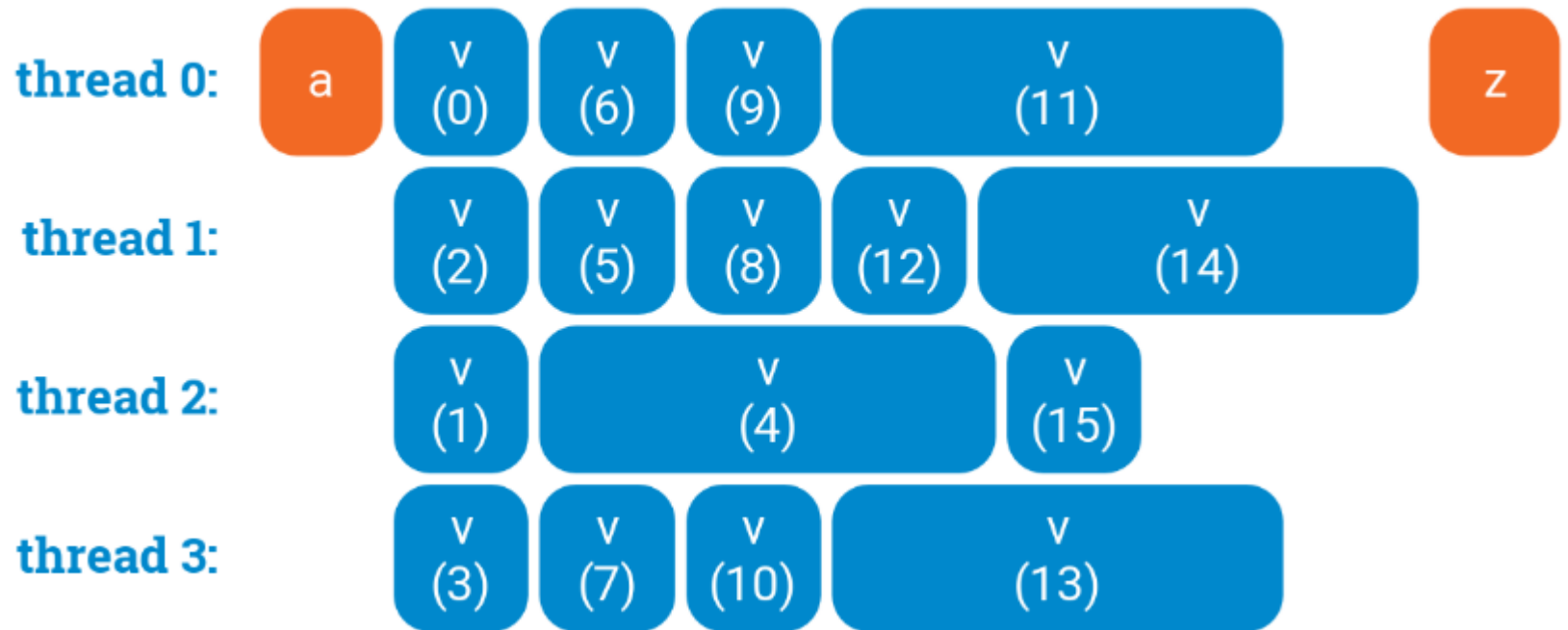


thread 3:



for schedule *dynamic*

```
a();  
#pragma omp parallel for schedule(dynamic,1)  
for (int i = 0; i < 16; ++i) {  
    v(i);  
}  
z();
```



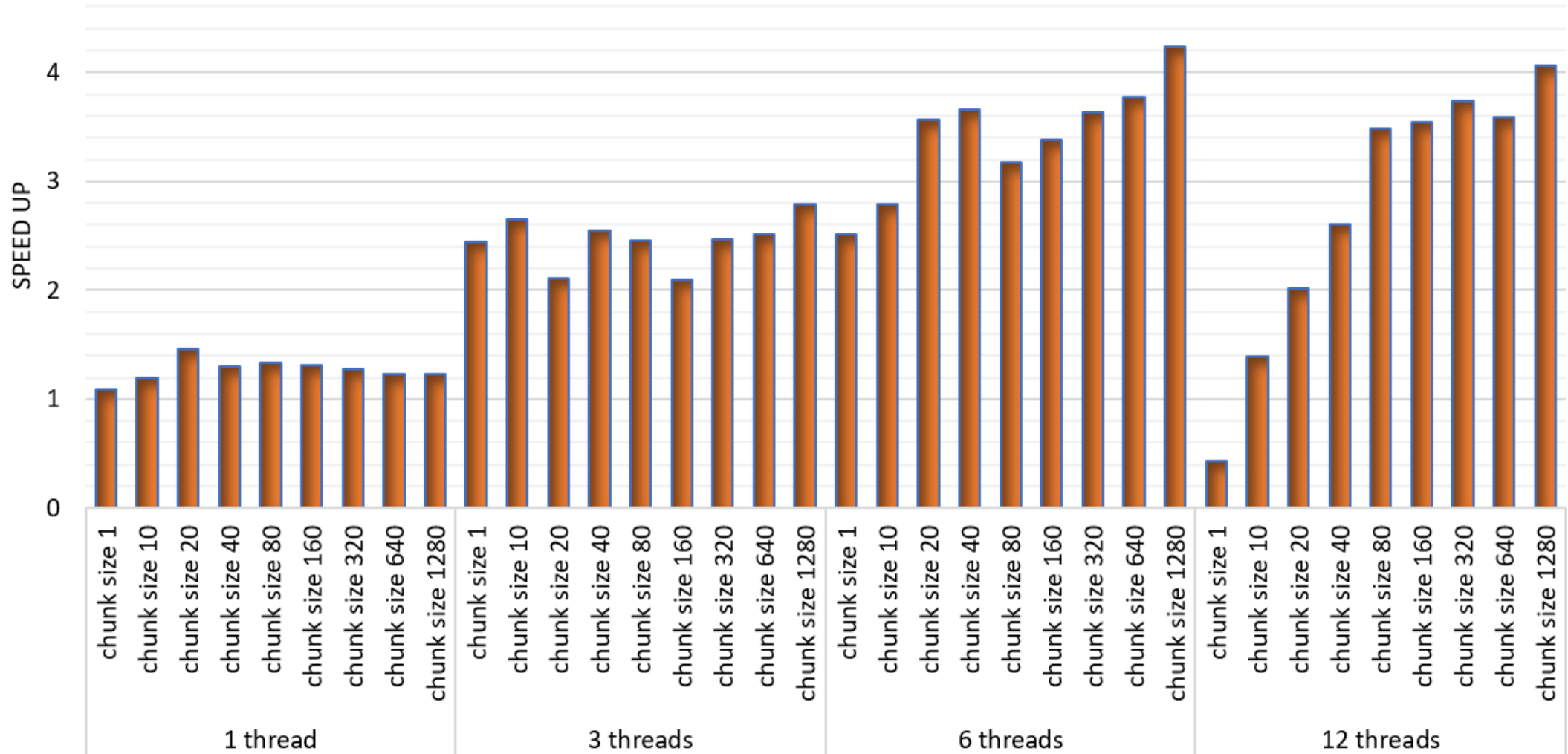
Experimento

Parametrizar el programa

- Tamaño del arreglo {1,000,000,000}
- ***For Normal***
 - Número de hilos {1, max/2, max}
- ***For Schedule Static***
 - Tamaño del chunk {1, 10, 20, 40, 80, 160, 320, 640, 1280}
 - Número de hilos {1, max/2, max}
- ***For Schedule Dynamic***
 - Tamaño del chunk {1, 10, 20, 40, 80, 160, 320, 640, 1280}
 - Número de hilos {1, max/2, max}

Experimento: Resultados

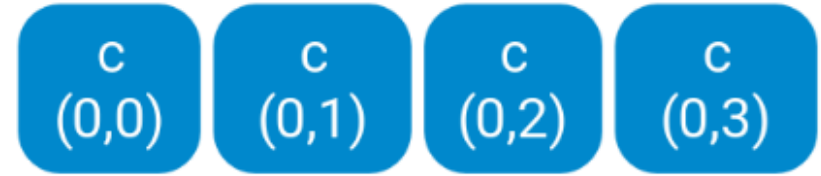
Asignación estática



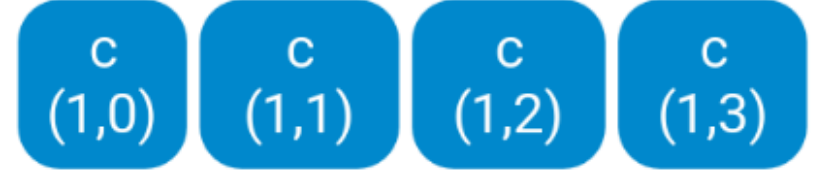
Fors anidados

```
#pragma omp parallel for
for (int i = 0; i < 4; ++i) {
    for (int j = 0; j < 4; ++j) {
        c(i, j);
    }
}
```

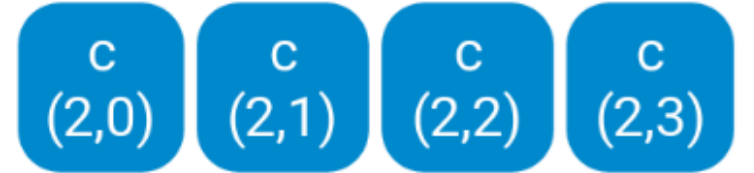
thread 0:



thread 1:



thread 2:



thread 3:

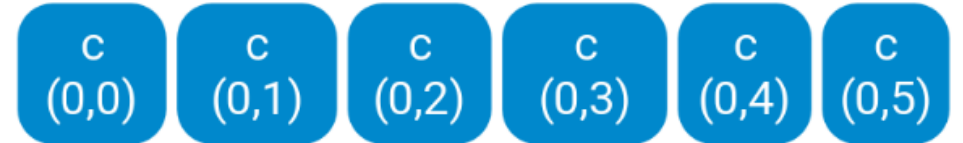


Fors anidados: Problema

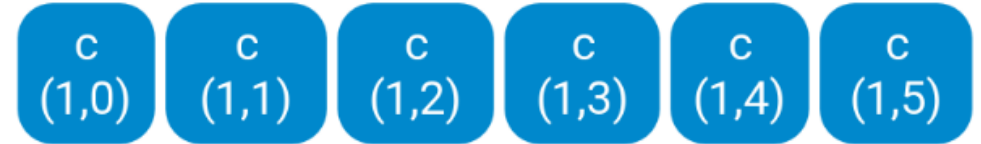
En ocasiones, no todos los hilos podrían utilizarse

```
#pragma omp parallel for
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
```

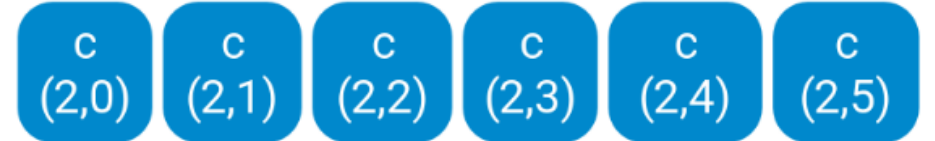
thread 0:



thread 1:



thread 2:



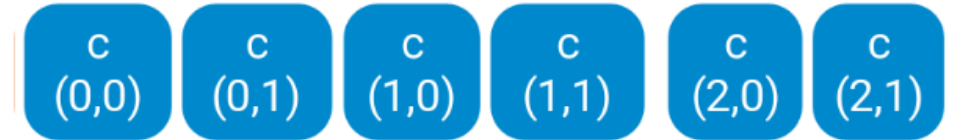
thread 3:

Fors anidados: Solución parcial

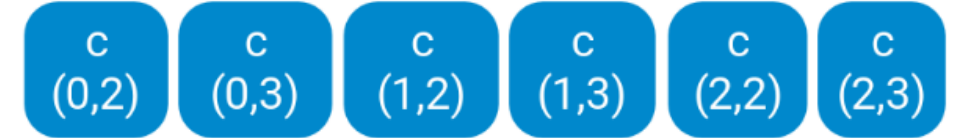
En ocasiones, no todos los hilos podrían utilizarse. **Posible solución:**

```
for (int i = 0; i < 3; ++i) {  
    #pragma omp parallel for  
    for (int j = 0; j < 6; ++j) {  
        c(i, j);  
    }  
}
```

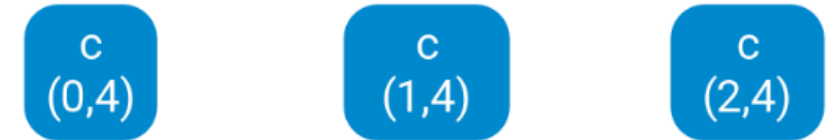
thread 0:



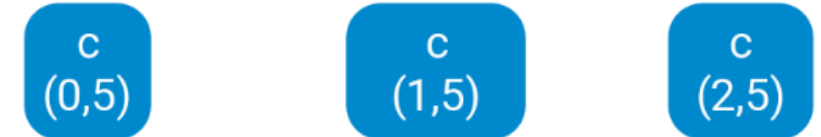
thread 1:



thread 2:



thread 3:

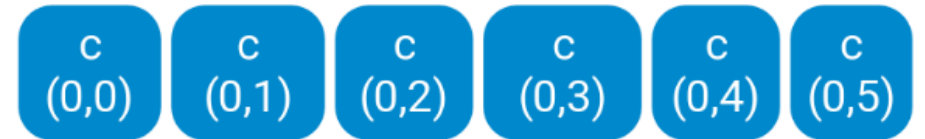


Fors anidados: El engaño

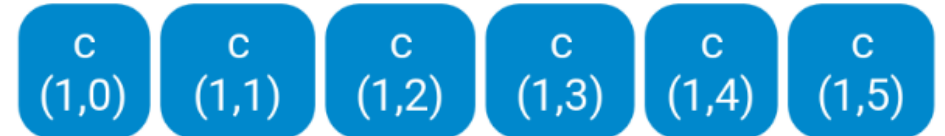
La forma incorrecta

```
#pragma omp parallel for
for (int i = 0; i < 3; ++i) {
    #pragma omp parallel for
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
```

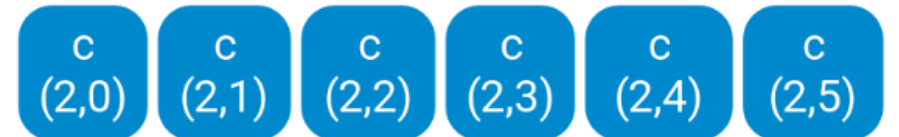
thread 0:



thread 1:



thread 2:

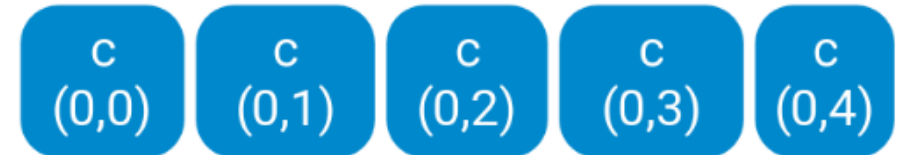


thread 3:

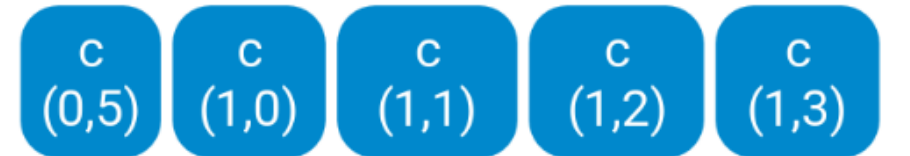
Fors anidados: Forma correcta 1

```
#pragma omp parallel for  
for (int ij = 0; ij < 3 * 6; ++ij) {  
    c(ij / 6, ij % 6);  
}
```

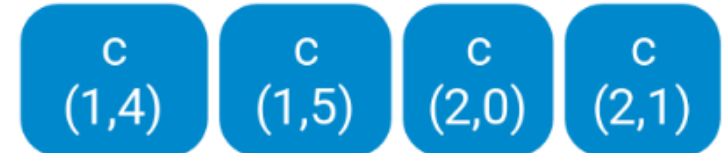
thread 0:



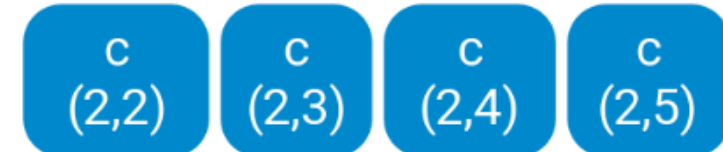
thread 1:



thread 2:



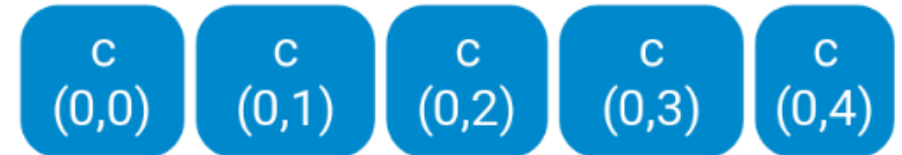
thread 3:



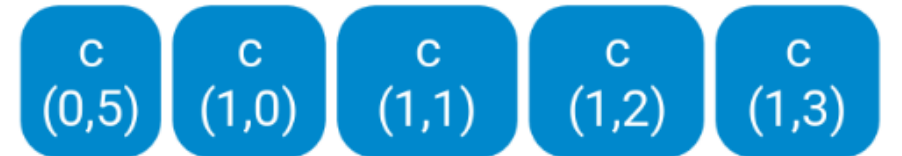
Fors anidados: Forma correcta 2

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
```

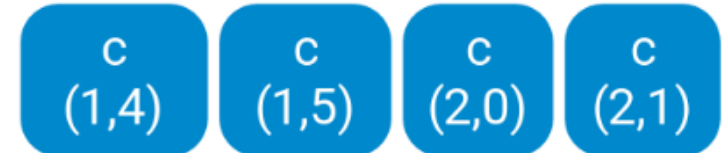
thread 0:



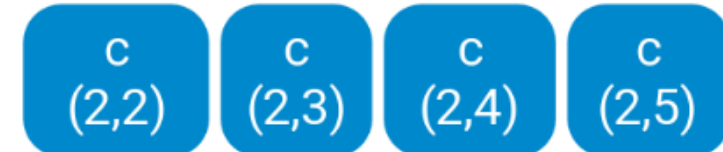
thread 1:



thread 2:

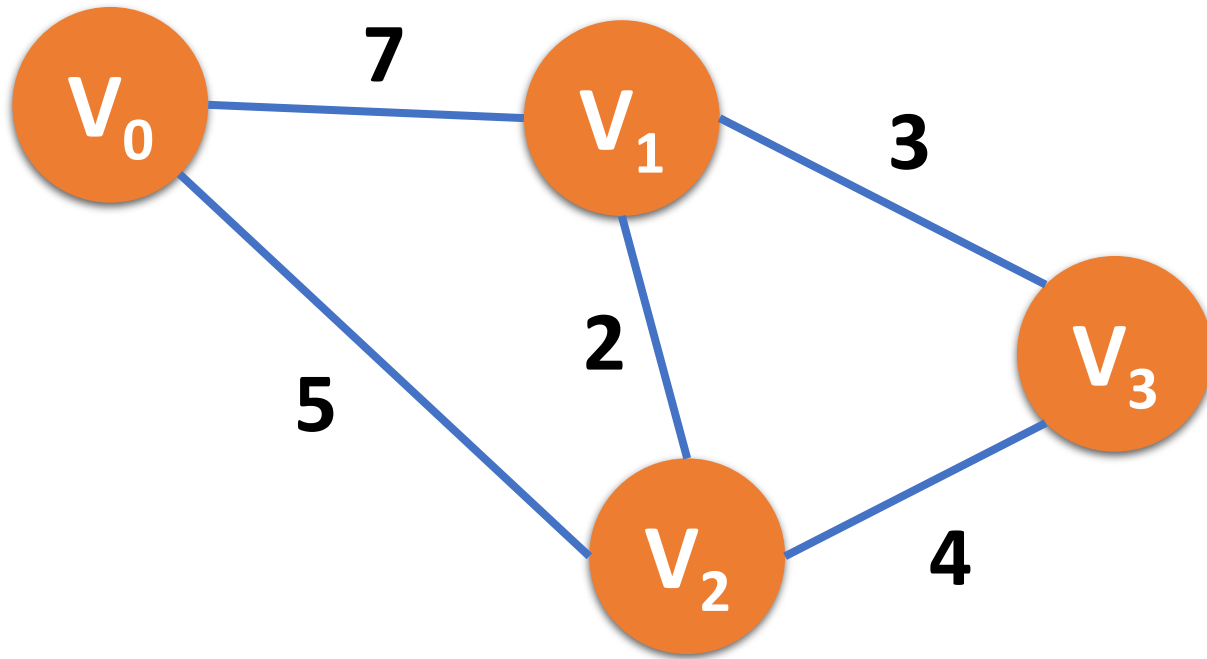


thread 3:



Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos

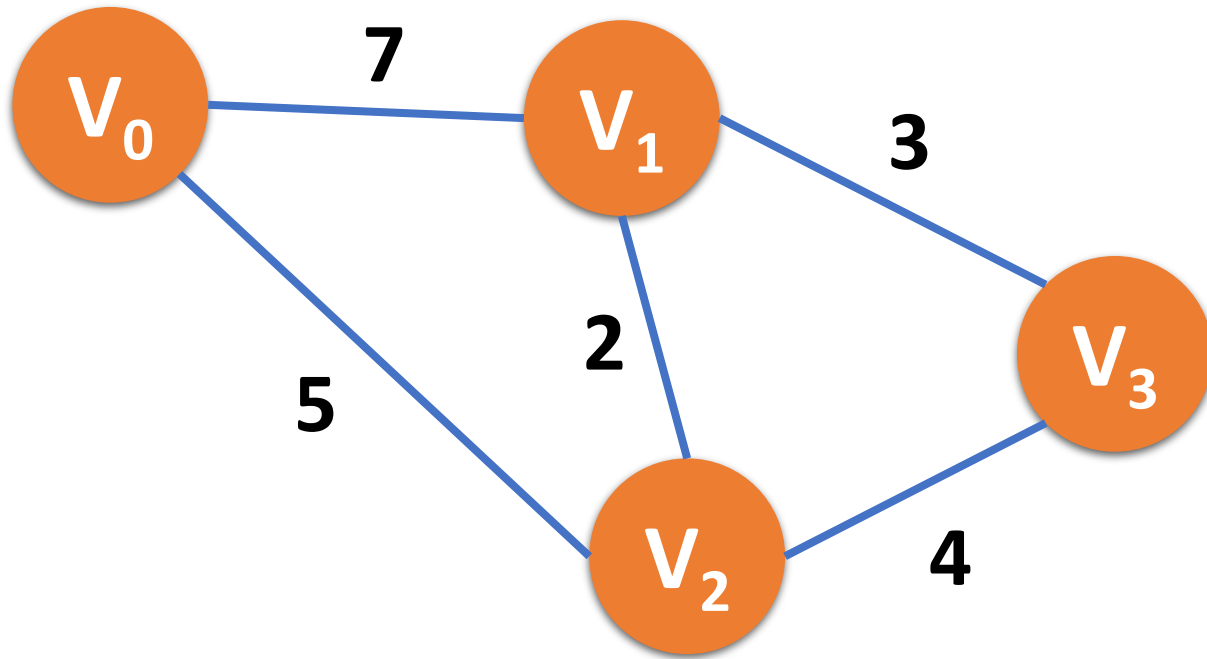


Para cada nodo k intermedio

Verifica si existe una ruta más corta entre cualquier par de nodos i, j a través de k

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos

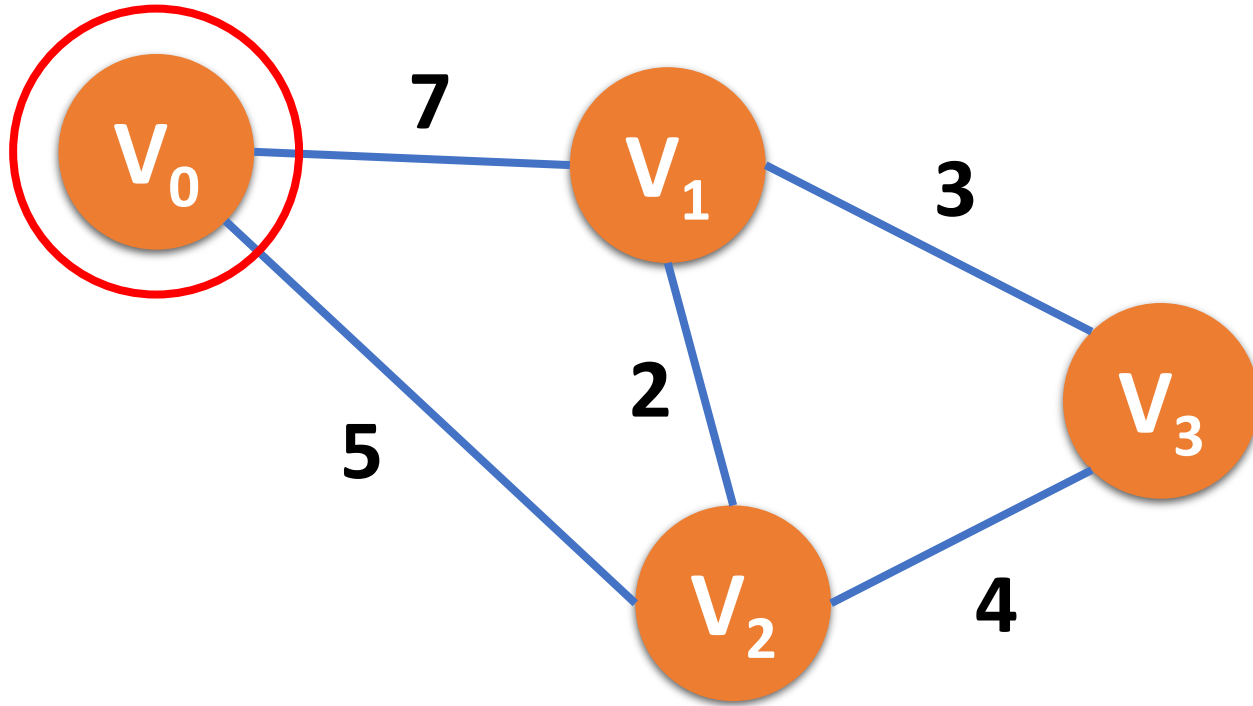


	V_0	V_1	V_2	V_3
V_0	0	7	5	0
V_1	7	0	2	3
V_2	5	2	0	4
V_3	0	3	4	0

Matriz de Adyacencia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



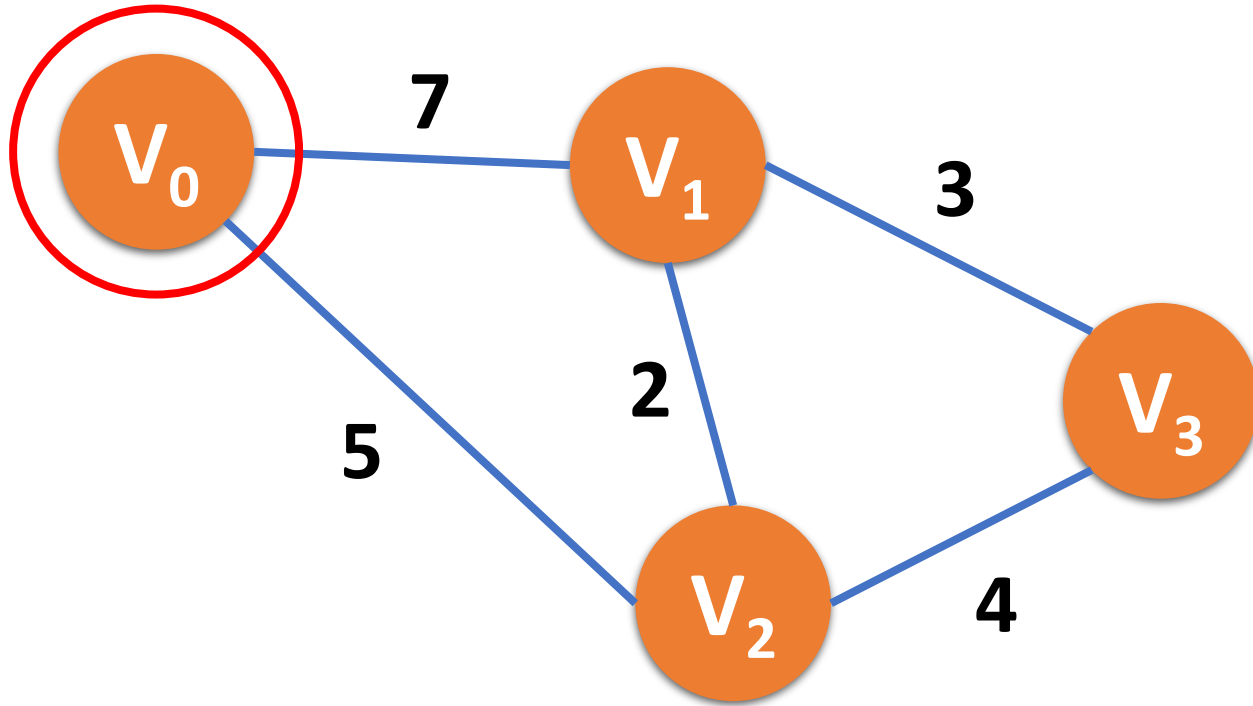
¿k = 0?

	V ₀	V ₁	V ₂	V ₃
V ₀	0	7	5	0
V ₁	7	0	2	3
V ₂	5	2	0	4
V ₃	0	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



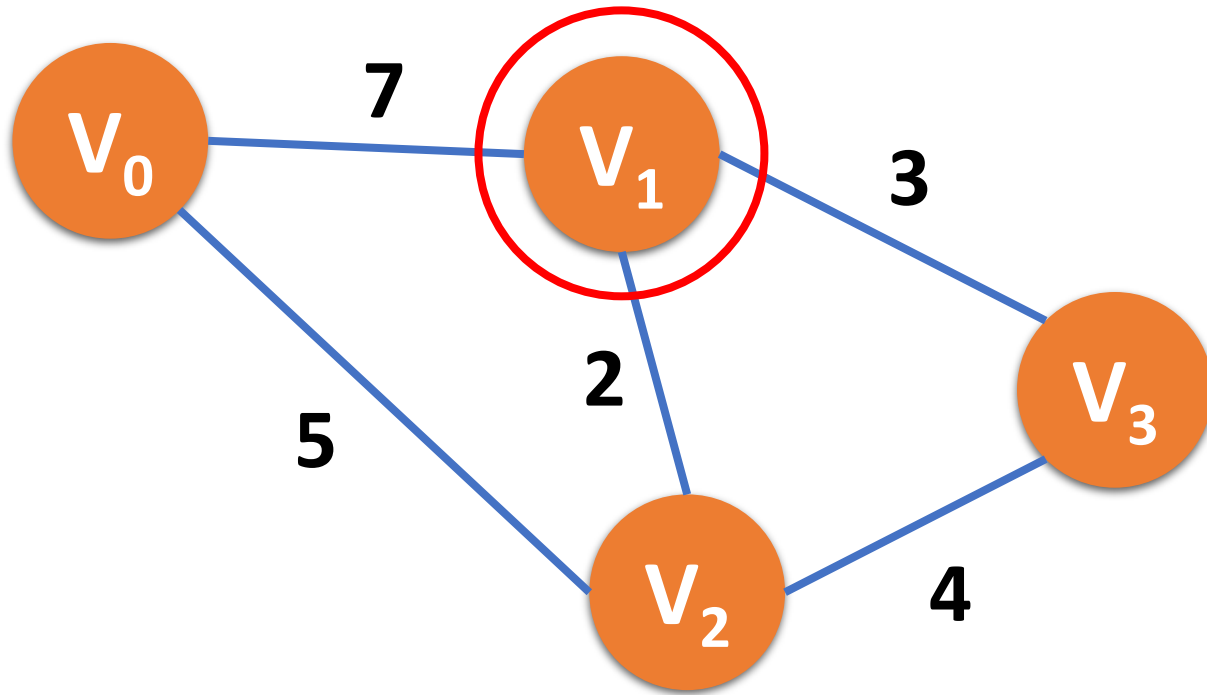
$k = 0$

	V_0	V_1	V_2	V_3
V_0	0	7	5	0
V_1	7	0	2	3
V_2	5	2	0	4
V_3	0	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



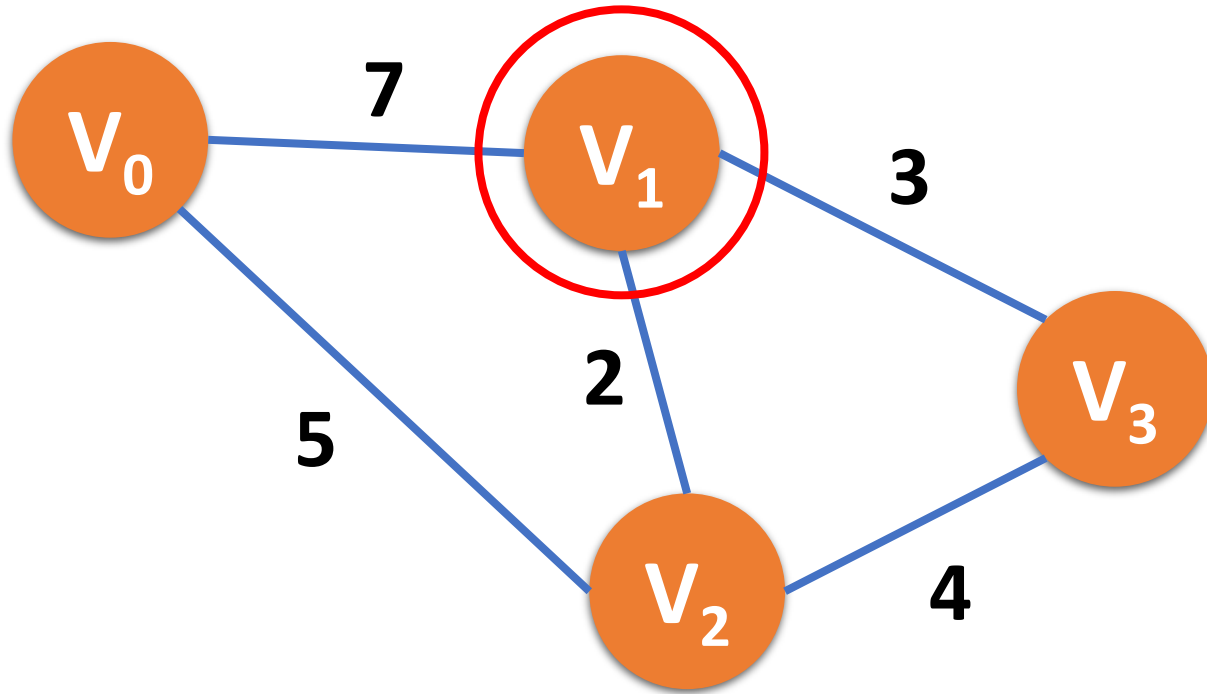
¿k = 1?

	V ₀	V ₁	V ₂	V ₃
V ₀	0	7	5	0
V ₁	7	0	2	3
V ₂	5	2	0	4
V ₃	0	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



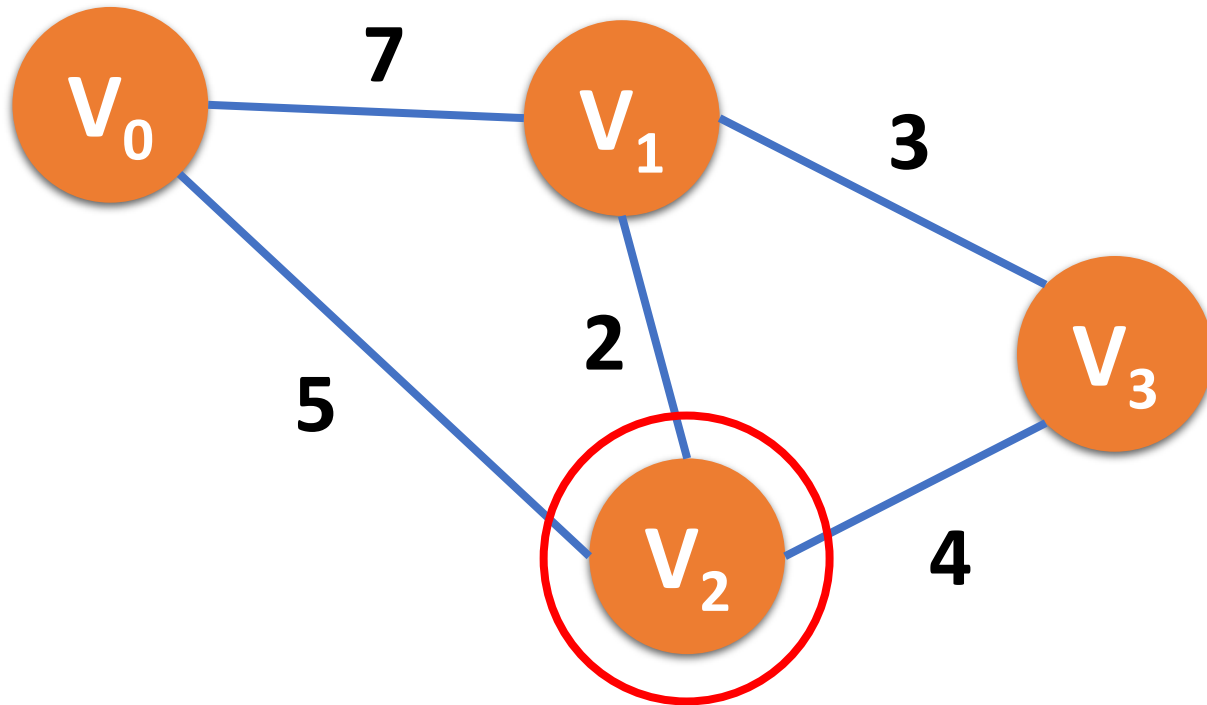
$k = 1$

	V_0	V_1	V_2	V_3
V_0	0	7	5	10
V_1	7	0	2	3
V_2	5	2	0	4
V_3	10	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



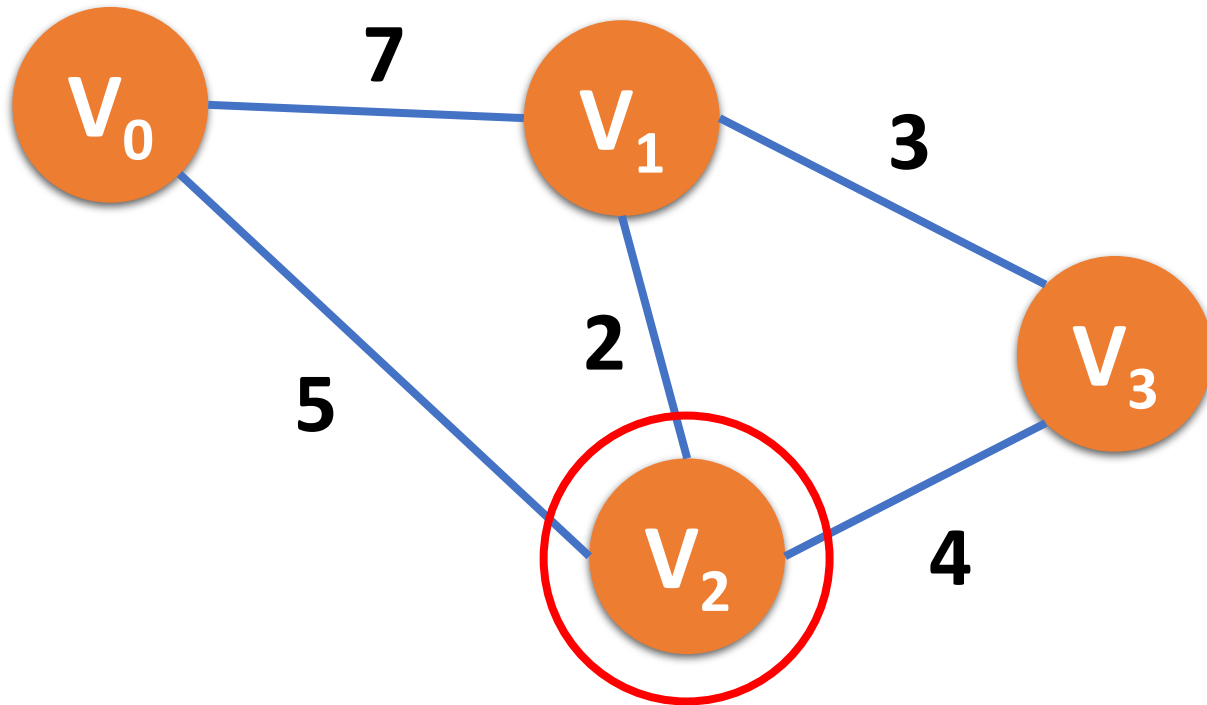
¿k = 2?

	V_0	V_1	V_2	V_3
V_0	0	7	5	10
V_1	7	0	2	3
V_2	5	2	0	4
V_3	10	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



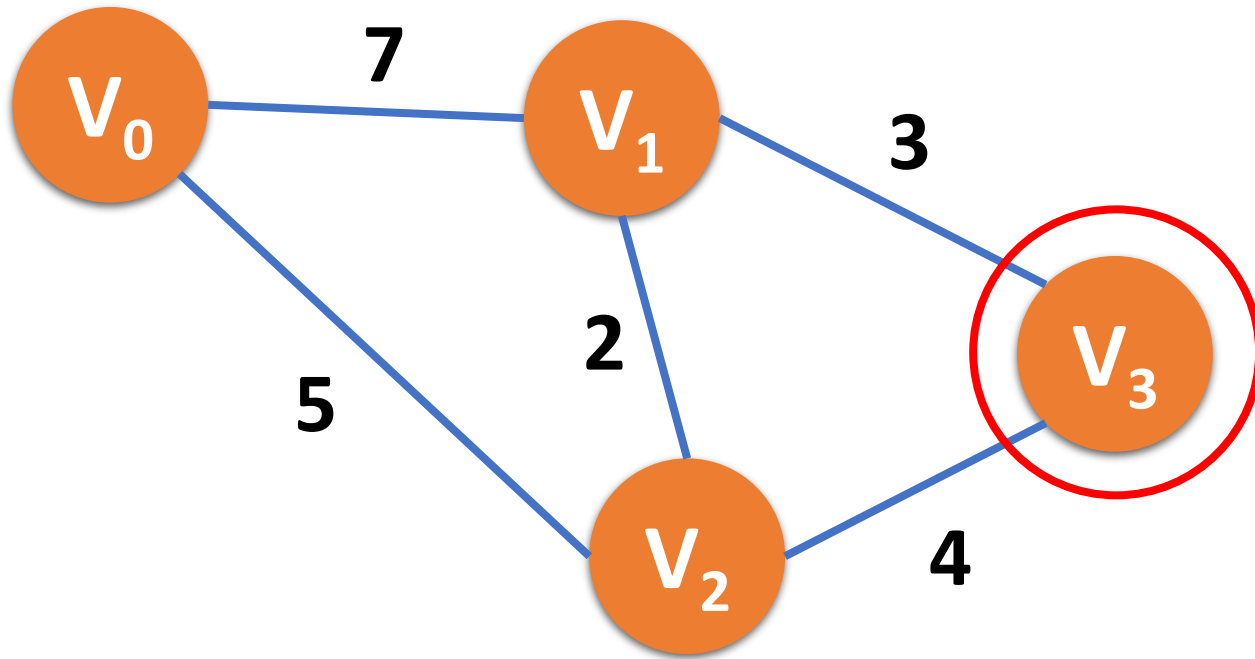
$k = 2$

	V_0	V_1	V_2	V_3
V_0	0	7	5	9
V_1	7	0	2	3
V_2	5	2	0	4
V_3	9	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



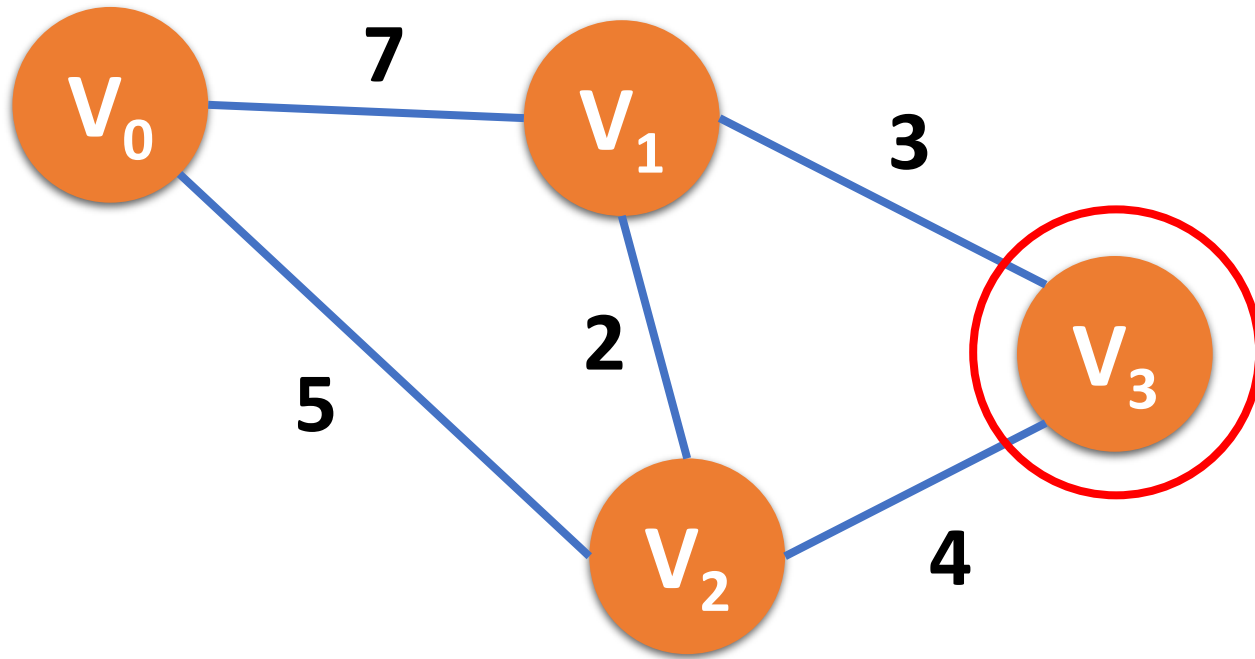
¿k = 3?

	V_0	V_1	V_2	V_3
V_0	0	7	5	9
V_1	7	0	2	3
V_2	5	2	0	4
V_3	9	3	4	0

Matriz de Distancia

Fors anidados: Floyd-Warshall

Rutas más cortas entre todos los nodos



$k = 3$

	V_0	V_1	V_2	V_3
V_0	0	7	5	9
V_1	7	0	2	3
V_2	5	2	0	4
V_3	9	3	4	0

Matriz de Distancia

Directiva **CRITICAL**

Región crítica



#pragma omp critical (*nombre*) **newline**
// structured_block

```
#pragma omp parallel shared(x)
{

    #pragma omp critical
    x = x + 1;

}
```

Directiva **CRITICAL**

```
int x = 0; // recurso compartido cuya actualización podría causar condición de carrera
#pragma omp parallel shared(x) private(i)
{
    #pragma omp critical (region_uno)
    {
        for (i=0; i<1000; i++){
            x = x + 1;
        }
    }
    #pragma omp critical (region_dos)
    {
        for (i=0; i<1000; i++){
            x = x + 1;
        }
    }
}

cout << "Valor de x = " << x << "\n";
```



Directiva **CRITICAL**



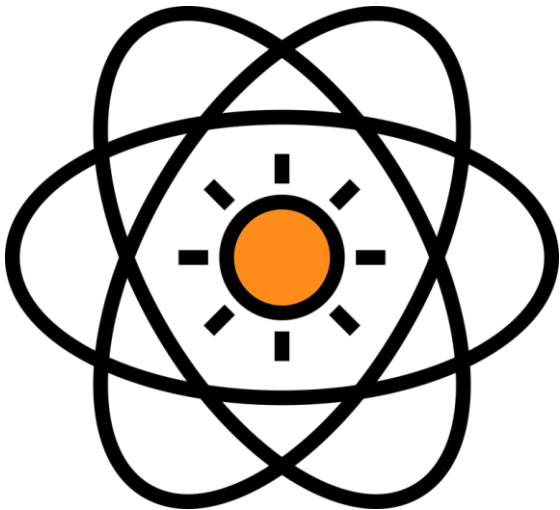
*El **(nombre)** es muy importante dado que funge como el candado.*

- *Regiones con el mismo nombre **son mutuamente excluyentes***
- *Regiones sin nombre son tratadas como **mutuamente excluyentes***
- *Para tener regiones que **no** sean mutuamente **excluyentes** se le debe poner un **nombre diferente** a cada región.*

Directiva **ATOMIC**

La expresión se ejecuta como si fuera una sola instrucción a nivel máquina.

#pragma omp atomic newline
// statement_expression



```
#pragma omp parallel shared(x)
{

    #pragma omp atomic
    x = x + 1;

}
```

Directiva **MASTER**

Bloque de código ejecutado exclusivamente por el hilo maestro

```
#pragma omp master newline  
// structured_block
```



Directiva **SINGLE**

Bloque de código ejecutado exclusivamente por un hilo

```
#pragma omp single newline  
// structured_block
```

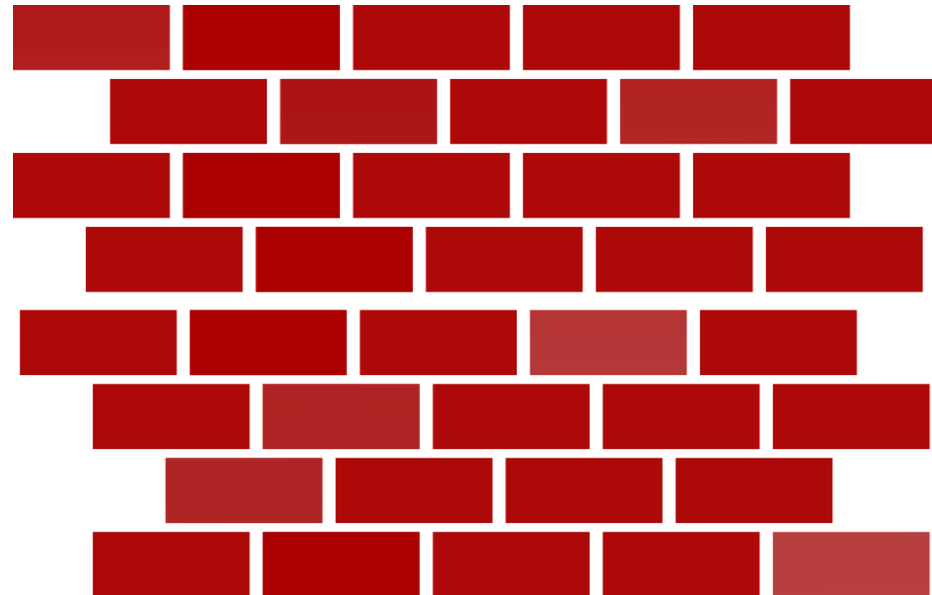
```
#pragma omp single nowait  
// structured_block
```



Directiva **BARRIER**

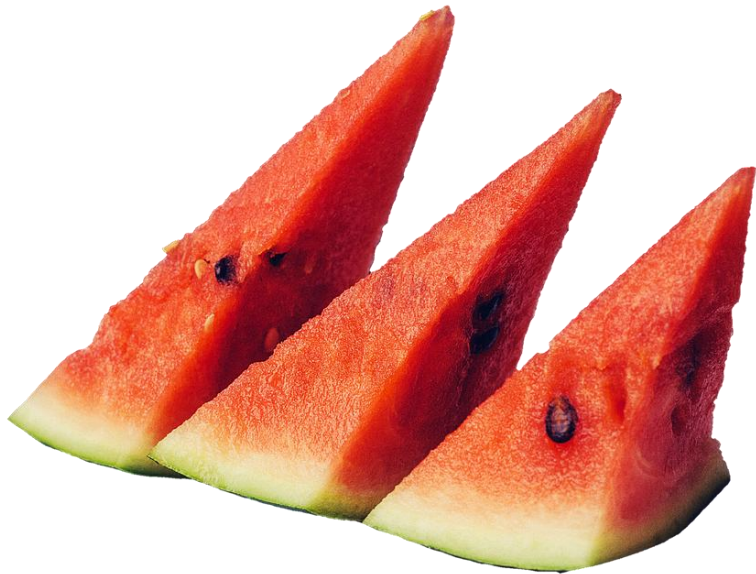
Obliga a todos los hilos a esperarse los unos a los otros

#pragma omp barrier newline



Directiva **SECTIONS**

Especifica que las secciones deben repartirse entre los hilos. Solo se permite un hilo por sección.



```
#pragma omp sections newline
```

```
{
```

```
    #pragma omp section newline
```

```
    // structured_block
```

```
    #pragma omp section newline
```

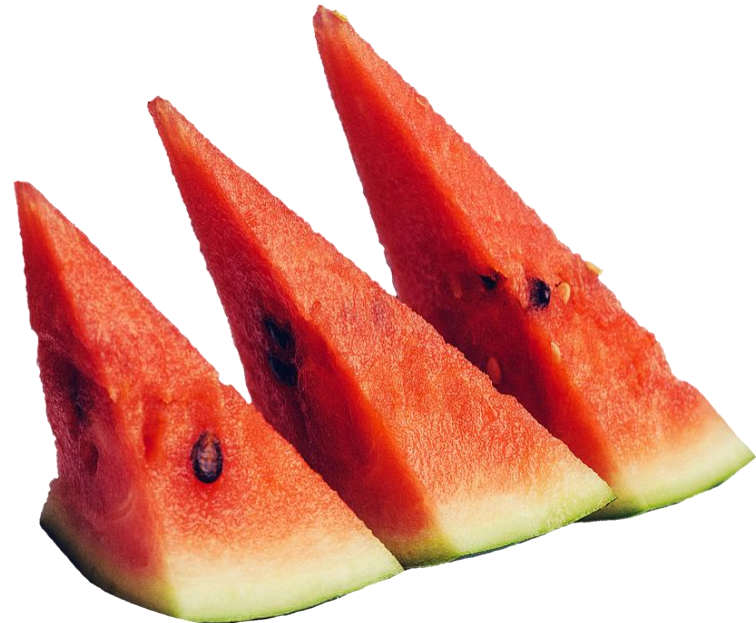
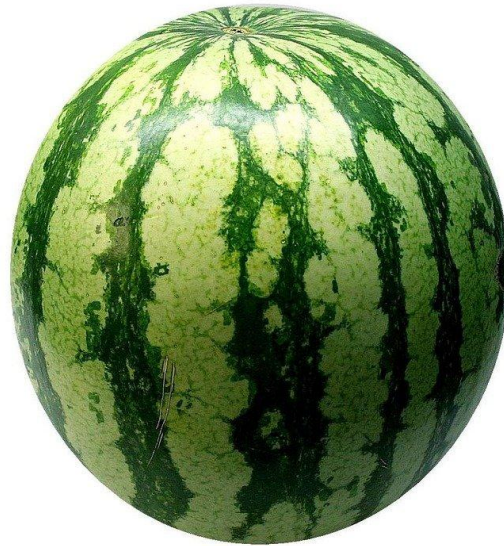
```
    // structured_block
```

```
}
```

Directiva **SECTIONS**

Diferentes secciones podrían ser ejecutadas por diferentes hilos.

*Al final de las secciones hay una barrera implícita a no ser que se utilice la cláusula **nowait***





Cláusula **REDUCTION**

#pragma omp for schedule(static, chunk_size) reduction(+:suma)

suma += expresión

#pragma omp for schedule(static, chunk_size) reduction(*:mult)

mult *= expression

Cláusula REDUCTION

Operadores

A nivel bit	+	<code>omp_priv = 0</code>
	-	<code>omp_priv = 0</code>
	*	<code>omp_priv = 1</code>
	&	<code>omp_priv = ~ 0</code>
		<code>omp_priv = 0</code>
A nivel lógico	^	<code>omp_priv = 0</code>
	&&	<code>omp_priv = 1</code>
		<code>omp_priv = 0</code>
	max	<code>omp_priv = Least representable number in the reduction list item type</code>
	min	<code>omp_priv = Largest representable number in the reduction list item type</code>

```
#include <random>
#include <iostream>

using namespace std;

int* generarVector(int n) {
    // reservar memoria
    int* v = new int[n];

    // generador aleatorio
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(1, 1000); // positivos

    // llenar vector
    for (int i = 0; i < n; i++)
        v[i] = dis(gen);

    return v;
}
```

```
int main() {
    int n = 10;
    int* vec = generarVector(n);

    for (int i = 0; i < n; i++)
        std::cout << vec[i] << " ";

    delete[] vec; // liberar memoria
}
```